

Headless Privacy Analytics - Architectural Plan

1. The Tiers

Tier 1: Cookieless Backbone (Default Mode)

This is the default tracking mode, fully compliant with GDPR and other strict privacy laws.

- **Data Collection Trigger:** Every page load and every custom event trigger.
- **Client-Side:** No cookies, `localStorage`, or `sessionStorage` are used. No unique identifiers are generated or stored.
- **Data Sent to API:**
 - `url`: The full URL of the page being viewed.
 - `referrer`: The `document.referrer`.
 - `screenWidth`: The width of the user's screen.
 - `eventName` (optional): For custom events like 'button-click'.
 - `eventData` (optional): A JSON object for custom dimensions.
- **Server-Side Anonymization (Crucial):**
 - **IP Address:** The server receives the IP. It is immediately used to create a **daily salted hash**.
 - `hash = hash_function(ip_address + daily_secret_salt)`
 - This hash allows us to count "unique visitors" within a 24-hour window without storing any PII. The salt must change every day, making it impossible to track a user across days.
 - The raw IP address is **never** stored.
 - **User-Agent:** The server receives the User-Agent string. It is generalized to its core components (e.g., "Chrome on Desktop," "Safari on Mobile") to prevent fingerprinting. The full, specific string is **never** stored.
- **Data Stored:**
 - `website_id`: Foreign key to the website being tracked.
 - `url`: The page URL.
 - `referrer`: The referrer URL.
 - `daily_ip_hash`: The anonymized IP hash.
 - `generalized_user_agent`: The simplified User-Agent.
 - `timestamp`: The time of the event.

Tier 2: Consent-Based Tracking (Opt-In Mode)

This mode is only activated after explicit user consent.

- **Activation Trigger:** A JavaScript function call from a Consent Management Platform (CMP).
- **Client-Side:**
 - A cryptographically random, long-lived `visitorId` is generated and stored in `localStorage`.
 - A `sessionId` is generated and stored in `sessionStorage`.
- **Data Sent to API:** Same as Tier 1, but with two additional fields:
 - `visitorId` (optional)
 - `sessionId` (optional)
- **Server-Side:** The server sees the IDs and can link sessions together to build user journeys. All other anonymization practices (IP, User-Agent) still apply.

2. Symfony API Endpoint (`/api/receive`)

- **Method:** `POST`

Request Body (JSON):

```
{
  "url": "[https://example.com/pricing](https://example.com/pricing)",
  "referrer": "[https://google.com/](https://google.com/)",
  "screenWidth": 1920,
  "eventName": "signup-click",
  "websiteToken": "abc-123-def-456",
  "eventData": {
    "plan_type": "pro",
    "button_color": "blue"
  }
}
```

-
- **Controller Logic:**
 1. Receive the request.
 2. Use a Data Transfer Object (DTO) with validation constraints to ensure data integrity.
 3. Find the `Website` entity based on the `websiteToken`. If not found, reject.
 4. **Perform security checks (whitelisting, rate limiting).**
 5. Dispatch a message containing the DTO and request details (IP, User-Agent) to a message queue using Symfony Messenger.
 6. Return a `202 Accepted` response. This confirms receipt without waiting for the data to be processed.

3. Security & Throttling

Domain Whitelisting

To ensure that data is only accepted from authorized sources, the API will perform domain whitelisting.

- **Mechanism:** When a website is registered, its domain (e.g., `example.com`) is stored in the `websites` table. The API endpoint will inspect the `Origin` or `Referer` header of incoming requests.
- **Validation:** If the origin of the request does not match the registered domain for the provided `websiteToken`, the request is rejected immediately with a `403 Forbidden` status.
- **Benefit:** This prevents unauthorized websites from sending fake data to an account and acts as a first-line defense against malicious bots.

Rate Limiting

To prevent abuse from a single source, the API endpoint should be rate-limited.

- **Mechanism:** Using Symfony's Rate Limiter component, a limit can be applied based on the incoming IP address. For example, allowing a maximum of 100 requests per minute from a single IP.
- **Benefit:** This helps protect the application from denial-of-service attacks and runaway scripts.

4. The JavaScript Snippet (`aggregate.js`)

- A lightweight, embeddable JS file for client websites.
- The API endpoint within the script must be configured to point to the user's self-hosted domain.
- It will automatically track a `pageview` on load.
- It will expose a global function for tracking custom events, e.g.,
`window.MyAnalytics.track('signup-click', { plan_type: 'pro' })`.
- It will contain the logic to switch between Tier 1 and Tier 2 based on consent.

5. Database Schema (Simplified)

- `websites`
 - `id` (PK)
 - `name`

- `domain` (Used for whitelisting)
 - `public_token` (UUID, used by the JS snippet)
- `page_views`
 - `id` (PK)
 - `website_id` (FK)
 - `url`
 - `referrer`
 - `daily_ip_hash`
 - `generalized_user_agent`
 - `created_at`
- `events`
 - `id` (PK)
 - `website_id` (FK)
 - `page_view_id` (FK, to link an event to a specific view)
 - `event_name` (e.g., 'signup-click')
 - `custom__data` (**JSON**, for storing custom dimensions)
 - `created_at`

6. Self-Hosting & Deployment (Open Source)

This project is designed to be an open-source, self-hostable solution, giving users full control over their analytics data.

- **Configuration:** All instance-specific settings are managed via environment variables (`.env` file). This includes:
 - `DATABASE_URL`: The connection string for the user's chosen database (e.g., PostgreSQL, MySQL).
 - `MESSENGER_TRANSPORT_DSN`: The connection string for the message broker (e.g., RabbitMQ, Redis).
 - `DAILY_SALT_SECRET`: A user-generated random string to ensure the privacy of IP hashes.
- **Recommended Deployment (Docker):** The primary method of installation will be through Docker and Docker Compose. A `docker-compose.yml` file will orchestrate the services:
 - **webserver service:** An Nginx, Caddy, or FrankenPHP container to handle incoming HTTP requests.
 - **app service:** A PHP-FPM container running the Symfony application (if not using FrankenPHP).
 - **db service:** A PostgreSQL or MySQL database container.

- **queue service:** A message broker container, such as RabbitMQ (recommended for reliability) or Redis.
- **worker service:** A container running the Symfony Messenger consumer (`php bin/console messenger:consume`) to process queued jobs.
- **Database Setup:** The database schema is managed via Doctrine Migrations. Users will run a simple command (`php bin/console doctrine:migrations:migrate`) to set up or update their database.

7. High-Throughput Data Ingestion (Queued Tracking)

To ensure the system can handle high traffic volumes without data loss or performance degradation, a message queue is a core part of the architecture.

- **Mechanism:**
 - The `/api/receive` endpoint validates the incoming data and immediately dispatches a `TrackEventMessage` to a message bus (like RabbitMQ).
 - The API responds with `202 Accepted` almost instantly.
 - Dedicated, background `worker` processes consume messages from the queue.
 - A message handler class contains the actual processing logic: IP hashing, User-Agent generalization, and writing the final, anonymized data to the database.
- **Benefits:**
 - **Performance:** The user-facing API is extremely fast, as it doesn't wait for database operations.
 - **Reliability:** If the database is temporarily unavailable, tracking requests are safely held in the queue and processed when the database is back online, preventing data loss.
 - **Scalability:** The number of workers can be scaled up or down independently of the web servers to match the processing load.